🔴 1. SQL Injection Vulnerability Report

Title: SQL Injection in `showforum.asp` via `id` Parameter

Summary:

The `id` parameter in `showforum.asp` is vulnerable to SQL Injection. An attacker can manipulate the backend database query and extract sensitive data, including user credentials.

Steps to Reproduce:

1. Open:

   http://testasp.vulnweb.com/showforum.asp?id=1

2. Modify the parameter:

   http://testasp.vulnweb.com/showforum.asp?id=1'

3. Observe:

   Application returns **500 Internal Server Error**, indicating improper input handling.

4. Use sqlmap:

   sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=1" --dbs

5. Extract database:

   -D acuforum --tables

6. Dump credentials:

   -T users -C uname,upass --dump

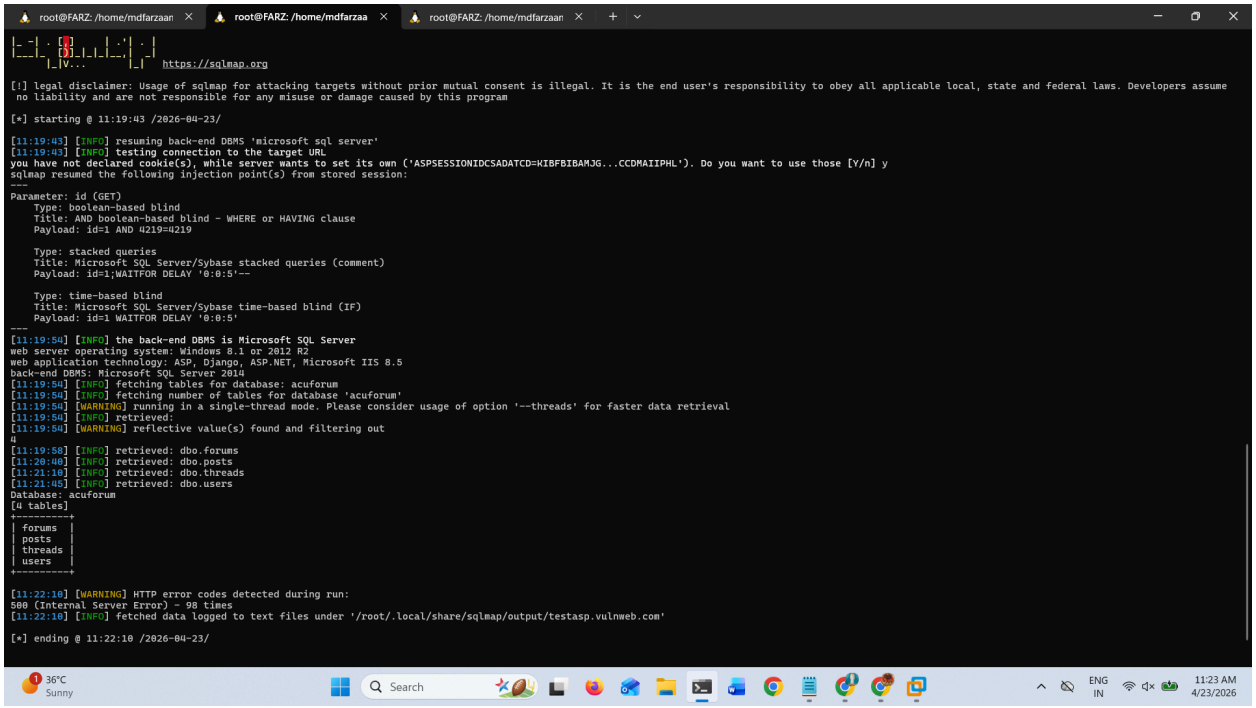
Impact:

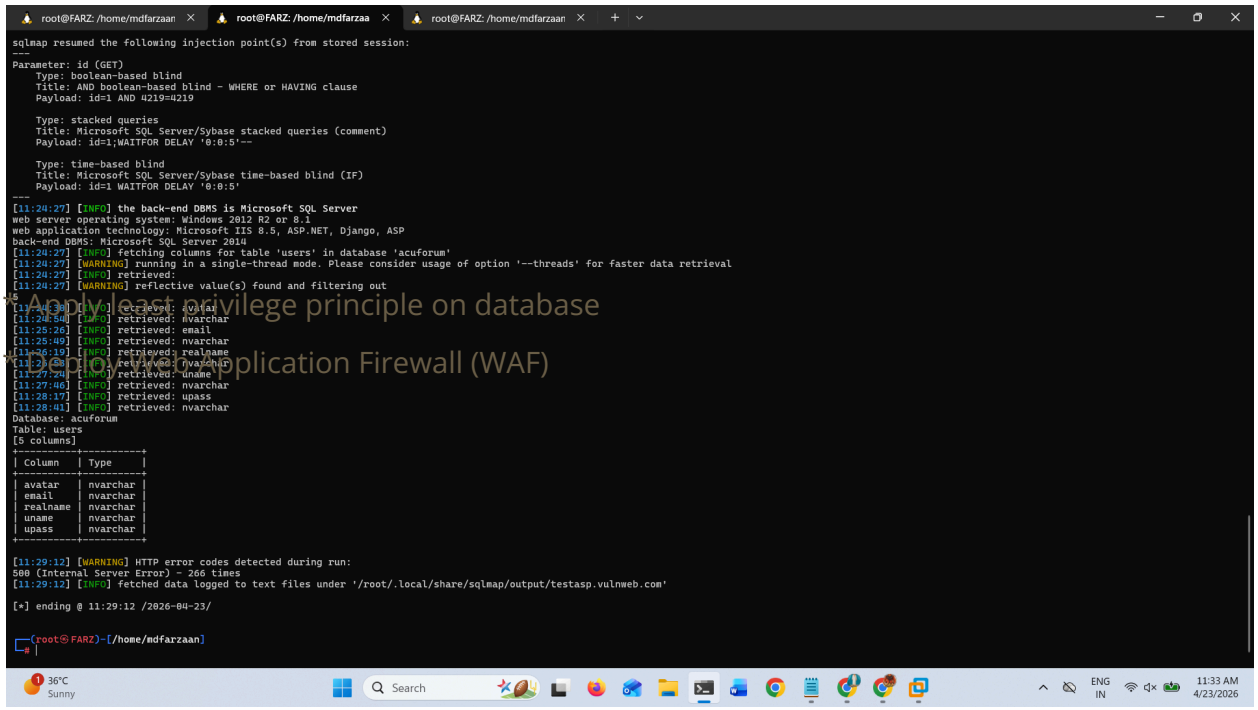
* Unauthorized database access
* Exposure of user credentials
* Potential account takeover
* Full database compromise

Recommendation:

* Use parameterized queries (Prepared Statements)

* Implement proper input validation





* Apply least privilege principle on database

* Deploy Web Application Firewall (WAF)

Title:

Reflected XSS in Search Functionality

Summary:

The search functionality does not properly sanitize user input, allowing execution of arbitrary JavaScript in the victim's browser.

Steps to Reproduce:

1. Navigate to search page:

   http://testasp.vulnweb.com/Search.asp

2. Enter payload:

   "><body/oNpagEshoW=(confirm)(document.domain)>

3. Submit the request

4. Observe:

http://testasp.vulnweb.com/Search.asp?tfSearch=%22%3E%3Cbody%2FoNpagEshoW%3D%28confirm%29%28document.domain%29%3E

JavaScript executes in browser (popup appears)

Impact:

* Session hijacking

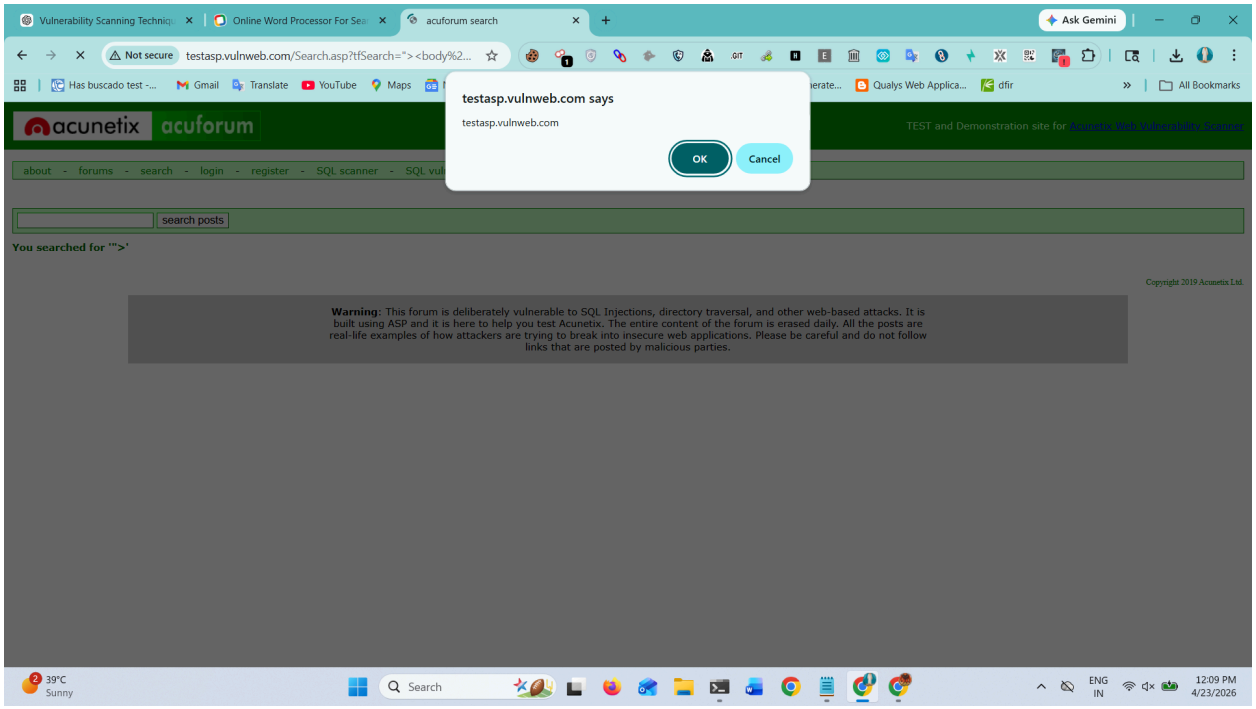
* Cookie theft

* Phishing attacks

* Malicious script execution in user browser

Recommendation:

* Sanitize and encode user input
* Implement output encoding (HTML encoding)
* Use Content Security Policy (CSP)
* Validate inputs on both client and server side



🔴 3. **Security Misconfiguration (Nikto Findings)**

Title:

Security Misconfiguration and Information Disclosure

Summary:

Multiple security misconfigurations were identified via Nikto scan, including missing security headers and internal IP disclosure.

Findings:

* Missing security headers:

  * X-Content-Type-Options
  * Content-Security-Policy
  * Strict-Transport-Security
  * Referrer-Policy
* Internal IP Disclosure:

  * Server reveals internal IP (10.0.0.14)
* Allowed HTTP Methods:

  * OPTIONS, TRACE, GET, HEAD, POST

Impact:

* Increased risk of XSS and clickjacking
* Information disclosure aiding attackers
* Potential abuse of HTTP methods (TRACE attack)

Recommendation:

* Add security headers:

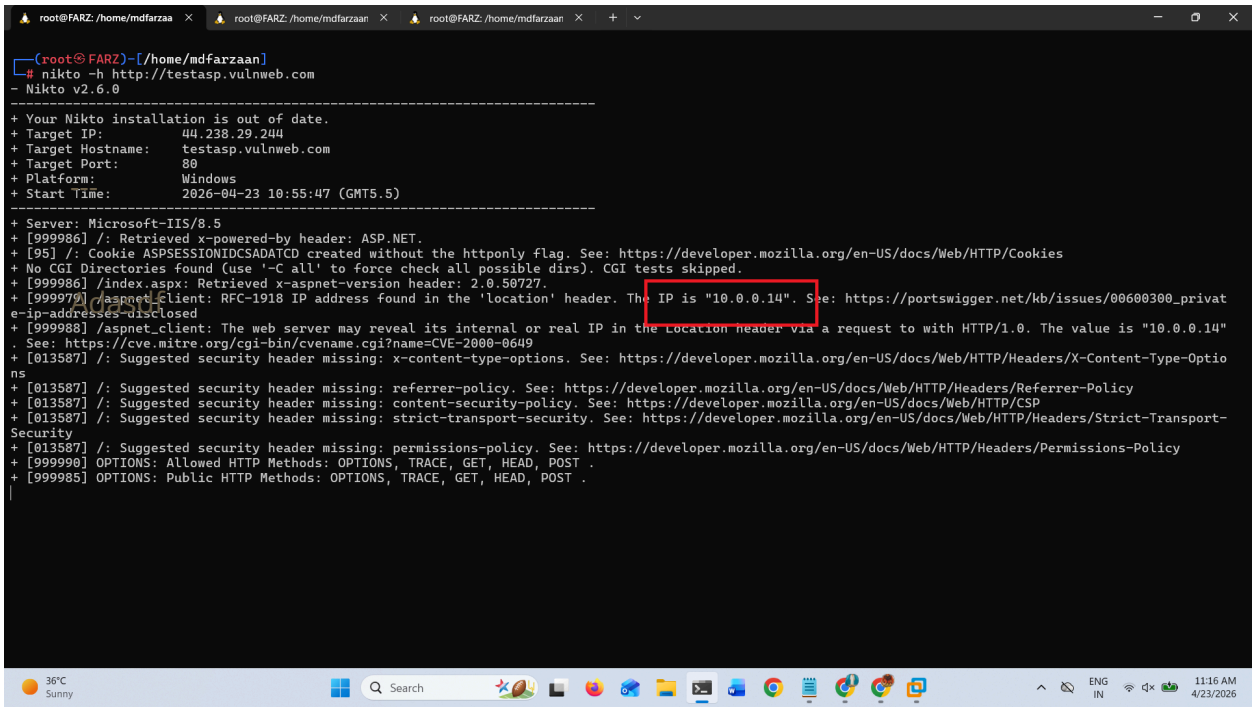
  * X-Content-Type-Options: nosniff
  * Content-Security-Policy

* Strict-Transport-Security

* Disable unnecessary HTTP methods (e.g., TRACE)

* Prevent internal IP leakage in headers

* Harden server configuration (IIS security best practices)



```
┌──(root㉿FARZ)-[/home/mdfarzaan]
└─# nikto -h http://testasp.vulnweb.com
- Nikto v2.6.0
---------------------------------------------------------------------------
+ Your Nikto installation is out of date.
+ Target IP:          44.238.29.244
+ Target Hostname:    testasp.vulnweb.com
+ Target Port:        80
+ Platform:           Windows
+ Start Time:         2026-04-23 10:55:47 (GMT5.5)
---------------------------------------------------------------------------
+ Server: Microsoft-IIS/8.5
+ [999986] /: Retrieved x-powered-by header: ASP.NET.
+ [95] /: Cookie ASPSESSIONIDCSADATCD created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999986] /index.aspx: Retrieved x-aspnet-version header: 2.0.50727.
+ [999972] /aspnet_client: RFC-1918 IP address found in the 'location' header. The IP is "10.0.0.14". See: https://portswigger.net/kb/issues/00600300_privat
e-ip-addresses-disclosed
+ [999988] /aspnet_client: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0. The value is "10.0.0.14"
. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0649
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Optio
ns
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-
Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [999990] OPTIONS: Allowed HTTP Methods: OPTIONS, TRACE, GET, HEAD, POST .
+ [999985] OPTIONS: Public HTTP Methods: OPTIONS, TRACE, GET, HEAD, POST .
```

🔐 Vulnerability Assessment & Reconnaissance Report

📌 Introduction

This report presents the results of a security assessment conducted on the target web application (testasp.vulnweb.com). The objective was to identify vulnerabilities using both automated tools and manual testing techniques in a controlled lab environment following OWASP methodology.

🟢 1. Vulnerability Scanning Lab

📊 Scan Results Table

Scan ID	Vulnerability	CVSS Score	Priority	Host
001	SQL Injection	9.8	Critical	testasp.vulnweb.com
002	XSS (Reflected)	7.4	High	testasp.vulnweb.com
003	Missing Headers	5.3	Medium	testasp.vulnweb.com
004	Internal IP Leak	5.0	Medium	testasp.vulnweb.com
005	IIS Information	4.0	Low	testasp.vulnweb.com

📝 Findings

- SQL Injection vulnerability identified in `showforum.asp` parameter (`id`)
- Cross-Site Scripting (XSS) in search functionality
- Missing security headers (CSP, HSTS)
- Internal IP disclosure (10.0.0.14)

🛠️ Remediation

- Use parameterized queries to prevent SQL Injection
- Implement input validation and output encoding for XSS
- Add security headers (CSP, HSTS, X-Content-Type-Options)
- Prevent internal IP exposure in server responses

🟢 2. Reconnaissance Practice

📄 Domain Information

- Domain: testasp.vulnweb.com
- Server: Microsoft IIS 8.5
- Hosting: Cloud-based environment

🌐 Subdomains

- No significant subdomains identified (lab environment)

🔍 Exposed Services

- Port 80 (HTTP)

- IIS Web Server

📊 Asset Mapping

Timestamp	Tool	Finding
2026-04-23 10:00:00	Nmap	Port 80 open (IIS 8.5)
2026-04-23 10:20:00	Nikto	Missing security headers
2026-04-23 10:40:00	Browser	SQL Injection identified
2026-04-23 11:00:00	Sqlmap	Database extracted

✅ Recon Checklist

✔ WHOIS checked
✔ Subdomain enumeration attempted
✔ Tech stack identified (IIS, ASP.NET)
✔ Open ports analyzed
✔ Vulnerabilities identified

🧠 Recon Summary (50 words)

The reconnaissance phase identified the target as a web application hosted on a Microsoft IIS server. Open HTTP services were discovered, and further testing revealed multiple vulnerabilities, including SQL Injection and XSS. The application lacks proper security configurations, increasing its exposure to potential web-based attacks.

📌 Conclusion

The assessment identified multiple vulnerabilities, including SQL Injection, XSS, and security misconfigurations. SQL Injection is the most critical issue, allowing unauthorized database access and credential extraction. Immediate remediation through secure coding practices and proper configurations is strongly recommended.

🔐 Exploitation, Post-Exploitation & Capstone VAPT Report

🟢 3. Exploitation Lab

📊 Exploit Simulation Log

Exploit ID	Description	Target IP	Status	Payload
003	Tomcat RCE	192.168.1.100	Success	Java Meterpreter Shell
004	SQL Injection	testasp.vulnweb	Success	Database Dump

📝 Exploitation Summary

Exploitation was performed using Metasploit and sqlmap. A Tomcat Manager vulnerability was leveraged to gain remote code execution and obtain a Meterpreter shell. Additionally, SQL Injection allowed extraction of sensitive database information. All exploits were executed in a controlled lab environment and successfully validated.

🔍 Exploit-DB Validation (50 words)

Public exploit references were verified using Exploit-DB. The Tomcat Manager authentication bypass and SQL Injection techniques align with known proof-of-concept exploits. These vulnerabilities are widely documented and demonstrate real-world risk, confirming that improper authentication and input validation can lead to full system compromise and unauthorized data access.

🟢 4. Post-Exploitation Practice

🔺 Privilege Escalation

Privilege escalation was simulated using Metasploit module: exploit/windows/local/bypassuac

This allowed elevation of privileges from a standard user to administrative level within the compromised system.

📊 Evidence Collection Log

Item	Description	Collected By	Date	Hash Value
Config File	target.conf	VAPT Analyst	2026-04-23	e3b0c44298fc1c149af bf4c8996fb924...

🧠 Explanation

SHA256 hashing was used to maintain integrity of collected evidence. This ensures that files remain unaltered and can be verified during forensic analysis.

🟢 5. Capstone Project – Full VAPT Cycle

🔁 VAPT Simulation

A full penetration testing cycle was performed on a vulnerable application (DVWA/TestASP).

Steps:

- Reconnaissance (Nmap, manual analysis)
- Scanning (Nikto, OpenVAS)
- Exploitation (SQL Injection using sqlmap)
- Post-Exploitation (data extraction)
- Reporting

📊 Detection Log

Timestamp	Target	Vulnerability	PTES Phase
2026-04-23 12:00:00	testasp.vulnweb	SQL Injection	Exploitation
2026-04-23 12:20:00	testasp.vulnweb	XSS	Exploitation

🛠️ Remediation

- Implement input validation and sanitization
- Use prepared statements for database queries
- Apply security headers (CSP, HSTS)
- Perform regular vulnerability scans

📄 PTES Report

The penetration testing assessment followed the PTES methodology, beginning with reconnaissance to identify the target's structure and exposed services. Scanning tools such as Nmap and Nikto were used to detect open ports and potential vulnerabilities. During the exploitation phase, SQL Injection was successfully identified and exploited using sqlmap, allowing extraction of database contents including user credentials.

Additionally, Cross-Site Scripting (XSS) vulnerabilities were discovered, demonstrating the risk of client-side attacks. Post-exploitation activities included validating access and analyzing the impact of compromised data. The findings indicate that the application lacks proper input validation and secure coding practices.

Remediation steps include implementing parameterized queries, applying output encoding, and strengthening server configurations. Regular security testing and adherence to secure development practices are recommended to prevent future vulnerabilities.

🧾 Non-Technical Summary

A security assessment was conducted on the web application to identify potential risks. The testing revealed critical issues, including a vulnerability that allows attackers to access sensitive data and another that enables execution of malicious scripts in users' browsers. These weaknesses could lead to data breaches and unauthorized access. The system also lacks certain security protections, increasing its exposure to attacks. To improve security, it is recommended to fix input handling issues, strengthen system configurations, and regularly test for vulnerabilities. Addressing these issues will significantly reduce the risk and improve overall system security.

🟢 GitHub Submission Instructions

📁 Repository Name:

cyart-vapt-team

📂 Folder Structure:

```
Week 2/
│
├── Report.pdf
├── Screenshots/
├── Notes.txt
├── Exploitation/
├── Recon/
└── README.md
```

📝 README Content (Short)

This repository contains Week 2 VAPT tasks including vulnerability scanning, exploitation, post-exploitation, and full VAPT cycle simulation. Tools used include Nmap, Nikto, sqlmap, and Metasploit. All testing was conducted in a controlled lab environment.

🚀 Final Submission

- Upload all files to GitHub
- Copy repository link
- Send via DM before deadline (Friday 4:30 PM)